



Securing Organisational Digital Identity: A Full-Lifecycle Approach to Distributed Key Management

Mariia Bakhtina¹ · Jan Kvapil¹ · Petr Švenda¹ · Raimundas Matulevičius²

Received: 16 December 2025 / Accepted: 16 June 2026 / Published online: 29 July 2026
© The Author(s) 2026

Abstract

Organisational Digital Identity (ODI) commonly relies on keys controlled by a single entity, creating a single point of failure and preventing organisations from cryptographically enforcing custom access policies. Migrating to fully decentralised identity infrastructures is costly and time-consuming, especially in regulated e-governance systems. This paper addresses these limitations through an Information Systems Engineering contribution: the design and experimental validation of a Distributed Key Management System (DKMS) that secures a centrally managed, PKI-based ODI without replacing the existing infrastructure. Rather than introducing new cryptographic primitives, the DKMS selects and composes existing threshold signature schemes to cover the complete key management lifecycle, from generation and operational usage through to post-operational archiving, recovery, and destruction. Partial custody through threshold signatures distributes control over an organisation's signing key among multiple semi-trusted representatives, while remaining fully backwards compatible with standard RSA and ECDSA verification. A semi-automated signer mechanism is introduced to enforce human-in-the-loop policies without prohibitive latency. The design is validated through a proof-of-concept implementation for the X-Road data exchange system, evaluated across RSA and ECDSA threshold groups of sizes 2-of-2, 3-of-5, and 11-of-15. Results show that RSA-based DKMS achieves round-trip times comparable to single-party hardware tokens, confirming DKMS-based signing as a practical, zero-trust alternative to single-controller identity management.

Keywords Organisational digital identity · Key management · Security · Zero trust · Distributed control · Threshold signatures

1 Introduction

Organisational Digital Identity (ODI) ¹ defines an organisation and its attributes for other entities through credentials. It enables trust between business partners and ensures the authenticity and confidentiality of cross-organisational data exchanges [68]. ODI commonly relies on centrally managed credentials and keys used in Public Key Infrastructure

(PKI). A centralised management introduces a single point of failure [11] in the system. As an alternative, decentralised Identity Management (IdM) based on distributed ledgers has been proposed [11]. Decentralised ODI enhances security and identity control. However, this approach requires a new governance framework and a shift of the participants to a new infrastructure. This process is time-consuming and may require legislative updates, especially in information systems used in e-governance like X-Road [55] and Gaia-X [31]. Moreover, the decentralised trust and IdM address external threats, while internal actors contribute significantly to data breaches through privilege misuse [66]. Thus, centralised control by internal actors is an open issue which threatens the authenticity of messages sent on behalf of ODI.

Regardless of the identity and trust model, organisations may use digital wallets or hardware security modules as a part of their Information System (IS) to store their identity's certificates and key material, with policies defining access rights [36]. Proprietary solutions like OpenID [26] control internal authorisation, determining who can initiate cross-

✉ Mariia Bakhtina
bakhtina@fi.muni.cz

Jan Kvapil
kvapil@mail.muni.cz

Petr Švenda
svenda@fi.muni.cz

Raimundas Matulevičius
rma@ut.ee

¹ Masaryk University, Brno, Czech Republic

² University of Tartu, Tartu, Estonia

¹ If not specified otherwise, we use *identity* to refer to ODI.

organisational data exchange on behalf of ODI. Meanwhile, some organisations prefer external trusted partners to manage their identity and key materials [36, 51, 64], removing access control from their IS, but this raises concerns about potential compromise or misuse. Based on [11, 36], we recognise the need for a more secure yet backwards compatible identity model, which would allow enforcing security through custom access policies (under what conditions are ODI-related cryptographic keys accessible/used) for information systems that rely on a centrally issued ODI.

A critical gap exists at the intersection of the Information Systems Engineering (ISE) community and formal cryptography: state-of-the-art cryptographic primitives (e.g., such as threshold signatures) are largely studied in isolation by the cryptographic community, while IS practitioners who face insider-threat and zero-trust requirements lack actionable guidance on how to compose and deploy these primitives in full-lifecycle identity management systems. This paper bridges that gap. Our goal is *not* to propose new cryptographic primitives; rather, it is to design, specify, and validate a complete key management system that brings zero trust [17] to ODI management. To do so, we employ an existing cryptographic primitive, threshold signatures, in backwards-compatible manner to real information systems. Assuming centrally issued PKI-based credentials, this paper considers the following research question: *how to secure an existing organisational digital identity through key management mechanisms to achieve zero trust?*

In this paper, we follow the design science research method [40]. We review the knowledge base of cryptographic and business mechanisms which help secure ODI with a focus on key management. As a result, we design an artefact of the Distributed Key Management System (DKMS) that secures a centrally issued ODI. The DKMS uses partial custody and threshold signatures to secure ODI in cross-organisational data exchange through zero trust principles. The proposed system ensures that messages signed on behalf of ODI originate from the organisation and are not created by a single trusted ODI custodian or a company representative. The DKMS design application and utility are evaluated through implementation for the X-Road data exchange system.

This paper significantly extends our earlier conference paper [8], which introduced only the pre-operational and basic operational phases of the DKMS, evaluated solely for RSA signatures with a fixed threshold group size. The following contributions are *entirely new* to this journal version:

1. **Full key-lifecycle DKMS design** (Sec. 3). The conference version [8] covered only key generation, distribution, and basic signing. This paper adds (i) *key backup* and *proactive refresh* to the operational phase, and (ii) *archiving*, *recovery*, and *destruction* for the

post-operational phase – phases that are mandatory for practical deployment but were absent from the original design. Together, these additions transform a proof-of-concept into a deployable, standards-aligned (NIST SP 800-57) lifecycle design.

2. **Decision-tree implementation guide** (Sec. 3, Fig. 3, Table 2). A structured decision framework is provided to help an organisation select the appropriate cryptographic mechanisms and practices for each phase of the key management lifecycle. The framework is structured around the primary requirement of whether a single master key pair must exist, and derives from that choice the concrete technologies and procedures applicable at each subsequent phase. This artefact does not exist in [8].
3. **Extended semi-automated signer mechanism** (Sec. 3, Sec. 4.3). The conference version [8] briefly mentioned automated signer applications enforcing time-based policies. This paper elaborates on this concept by introducing a taxonomy of signer automation levels: fully automated signers, human signers, and semi-automated signers. The semi-automated signer contributes its partial signatures automatically, conditioned on, for example, recent human authentication. This is defined at the general DKMS design level and applied in the X-Road scenario to illustrate how a human-in-the-loop policy can be enforced without incurring the latency of real-time human approval.
4. **Extended multi-algorithm, multi-group experimental evaluation** (Sec. 4.3.4, Table 3). The original valuation [8] measured RSA performance for a single threshold configuration. This paper adds: (i) ECDSA threshold signatures; (ii) three threshold group sizes (2-of-2, 3-of-5, 11-of-15); and (iii) payload size analysis for both key generation and signing rounds. These results provide the first side-by-side feasibility comparison of RSA and ECDSA threshold signatures in a deployed data exchange system.
5. **BPMN process integration and extended background study** (Sec. 2, Sec. 4.3, Figs. 5–6). The conference version [8] described the X-Road testing scenario in text only. To clarify how the DKMS integrates into organisational business processes and to make the design more accessible to IS practitioners, this paper adds BPMN diagrams illustrating both the share distribution process and the threshold signing process. The background study is extended with coverage of post-operational key management mechanisms which are necessary to support the full-lifecycle DKMS design introduced in this paper. The related work section is updated accordingly to reflect these additions (Sec. 5).

By covering these additional phases and operations, this paper provides a full lifecycle design for DKMS-based identity management systems, which is crucial for their practical

adoption. The proposed DKMS can be adopted directly in existing PKI-based systems without requiring new cryptographic algorithms or infrastructure changes, making it directly applicable to the needs of the ISE community.

The rest of the paper is organised as follows. Section 2 introduces the background of organisational identity management, presenting a review of cryptographic and business mechanisms which help secure ODI with a focus on key management. In Section 3, we present the proposed Distributed Key Management System (DKMS) that secures an ODI. In Section 4, we demonstrate how the proposed design can be implemented and evaluate it for the case of X-Road data exchange system. Section 5 reviews the related work, while Section 7 concludes the work.

2 Background

This section builds upon and extends the background from our earlier work in [8]. To provide new services, multiple organisations may connect their systems to form a new information system. Securing such a system requires identifying the individual organisations to allow authenticity and integrity of the shared data. Data exchange systems enable secure interoperable cross-organisation data exchange between separately built information systems in e-governance and other industries [26, 49, 50]. The examples of commercial, worldwide used systems are X-Road [55], UXP [20], Dawex Data Exchange Platform, and European Gaia-X [31]; while Dutch NLX [67] and Australian Secure Data Exchange application are national. Yet, they operate securely under the assumption of already established trust between collaborating entities and their digital identities.

Digital identity enables participants of the data exchange systems to confirm the authenticity of involved entities. Credentials associated with identities are the documents enabling the verification, with issuance procedures determined by a trust model. While the procedures of issuing credentials in decentralised and centralised identity models vary [11], PKI stays essentially the same. Thus, the key management is equally relevant regardless of the trust and identity model. Both the internal and external trusted representatives that manage the keys may become malicious. To mitigate this threat, the organisation should control every single data exchange attempt made on its behalf. Such a mitigation strategy [17] of avoiding the need for trust is the key component of the Zero Trust (ZT) architecture [60]. Among the key components of ZT architecture is the policy decision point that enforces control over access to resources based on the defined policies and is supported by PKI and identity management, on which this work is focused.

2.1 Targeted System Characteristics

As zero trust is a maturing strategy, little research has been done to map which of the existing security mechanisms can help secure ODI with respect to it. To study the security of an ODI and Identity Management (IdM) system, we consider characteristics targeted by identity and key management mechanisms. The need for the characteristics depends on the scenario. Through the literature review, we gathered a set of non-functional characteristics that affect ODI's security. In [9], we describe the literature procedure and provide a mapping of system characteristics with reviewed in Sec. 2.2 mechanisms.

The following business-oriented characteristics may be targeted, primarily reflecting the trust among the individual business entities. *Trustlessness* is a part of the zero trust paradigm and refers to the ability of the system to operate without relying on the honest behaviour of internal or external entities [51, 60]. To react to the dishonest behaviour, the IdM system may target to deliver *traceability* that refers to the ability to know who did what, when, and how [23, 36]. The more proactive approach to secure the system against internal attacks is *privilege escalation prevention*, which aims to restrict users from gaining unauthorised access. In this research, we consider privilege escalation as a result of both privilege escalation attacks and privilege misuse. Another way of enabling zero trust is *decentralisation*, which refers to distributing the responsibility for managing ODI and its keys across multiple entities to enhance security, resilience, and user control. As a result, the organisation can differentiate between *multiple users* of the ODI. Another business-demanded aspect of its system's security is *availability*, which is the ability to deliver its value continuously. System availability [36] is crucial both in the everyday data exchange and in the case of extraordinary trigger events – e.g., a loss of access to the keys or if an employee that is involved in the ODI management leaves or becomes malicious. Finally, the *usability* refers to the feasibility and convenience of the identity and key management. Usability encompasses the backwards compatibility of the proposed design with the existing infrastructure. Besides, usability defines whether the end-users will use the ODI following the defined interaction setup with ODI [36, 51, 64].

Digital signatures are the primary cryptographic method for ensuring data authenticity and integrity. Key characteristics include *unforgeability*, which prevents unauthorised entities from creating a valid signature without the private key [23, 65]. This property, in turn, provides *non-repudiation* – as long as the key is secure, its owner cannot plausibly deny signing the data. Furthermore, schemes like ECDSA [43] and RSA [59] link signatures created with the same private key, a property known as *linkability*. This feature allows a single public key to verify multiple signatures from the same iden-

tity [23, 27], which can be either a desirable or undesirable trait depending on the application. Ultimately, these three characteristics – unforgeability, non-repudiation, and linkability – are fundamental to guaranteeing the authenticity and integrity of data.

2.2 Review of Key Management Mechanisms

Assuming ODI relies on PKI, key management is an enabler of digital identity per se and ensures the security of IdM systems. The key management includes pre-operational, operational, post-operational and destroyed phases [14]. IdM is involved in the four stages: (i) generation and distribution of asymmetric key pairs (i.e. private and public keys) for the key generation during the pre-operational phase; (ii) the key registration and the public key certification during the pre-operational phase; and (iii) storage, usage and backup of keys during the operational phase. The certification is crucial for establishing trust within the system as it enables verification of the exchanged data integrity. Our research aims to comply with the existing verification process to remain IdM backwards compatible. The DKMS assumes that PKI is already established in the targeted information system and that the registration process and its trust model cannot be changed. The DKMS is therefore designed to achieve zero trust within this constraint, ensuring that the threshold signatures it produces remain verifiable against the existing signature verification routines. Thus, we exclude the second stage from our analysis, but describe it briefly below for completeness of the key lifecycle picture.

Fig. 1 depicts artefacts used for key management that are discussed in this section. Here, we provide an overview of the cryptographic and business mechanisms found through the literature review for the pre-operational and operational phase, details of which can be found in [9]. Meanwhile, the procedural mechanisms for the post-operational key management phase are based on the NIST SP 800-57. This paper focuses on asymmetric cryptographic schemes that provide digital signatures.

A digital signature scheme is defined by key generation, signature creation and signature verification algorithms.

2.2.1 Pre-operational Key Phase

Key generation To use any of the existing signature schemes, a (master) key pair needs to be generated. The signature schemes differ by key and signature size and mathematical structure, computational efficiency and security assumptions. Regardless of the scheme, the key pair creation process involves a cryptographically secure pseudorandom number generator that needs to be seeded. At the expense of security, the seed can also be derived from a user-provided input, such as a password (using a password-based key derivation

function). To improve trustlessness and resilience and not rely on a single factor, the multi-factor key derivation function (MFKDF) [52] has been proposed. However, MFKDF has been later shown to be vulnerable to several attacks [62]. These issues are claimed to have been addressed in the successor version MFKDF2².

The key generation is commonly executed by a trusted dealer. Such a dealer can be either the key holder themselves or a trusted party (e.g., a company representative or 3rd-party service provider) who generates and distributes the key material (i.e., the private key in traditional single-party or key shares in threshold signature schemes, e.g., [63]). Also, there are protocols for Distributed Key Generation (DKG), such as FROST [47], where multiple parties collaboratively generate secret shares of a key in a distributed manner. DKGs have a major benefit over a trusted dealer – the key never exists in a complete form at a single location. Finally, once generated, the master (public) key can have a certificate issued that binds it to a particular entity.

Key distribution Upon the key generation, a trusted dealer distributes the key to its controller. The key distribution phase defines a point of control over the ODI's keys and their delivery to controllers. The point of control is driven by the level of custody over the keys. Non-custodial (i.e. self-custodial) approach [45] enables more control but more overhead, as the ODI owner is also the controller responsible for securing the keys. An opposite approach is full outsourcing of the control (i.e. custody) [51], which removes overhead from the ODI owner but disables control. Custodial ODI directly conflicts with the ZT strategy as it puts complete trust in a third-party custodian. The custodian manages the keys and can conduct any operations on behalf of the organisation. Thus, none of the approaches address trustlessness, resilience, or protection from privilege escalation and enable multiple conditional users using internal organisational access control. Another option is to distribute the custody over the master key using Shamir Secret Sharing (SSS). The trusted dealer can use SSS to split the key into parts and assign these to distinct custodians. Depending on the configuration, K -out-of- N custodians are then needed to reconstruct the master key before use. However, this also means that whenever the key is to be used, it exists in a single location and is more susceptible to a compromise.

The key generation and key distribution phases form a key establishment stage. After the key establishment is complete, the key may be used either right away, and move to the operational phase, or the additional registration and sub-key derivation phases may be executed.

Registration Upon the (master) key being established within the key generation and distribution phases, such a key might

² See <https://multifactor.com/blog/introducing-mfkdf2-mfchf2-mfdpg2>.

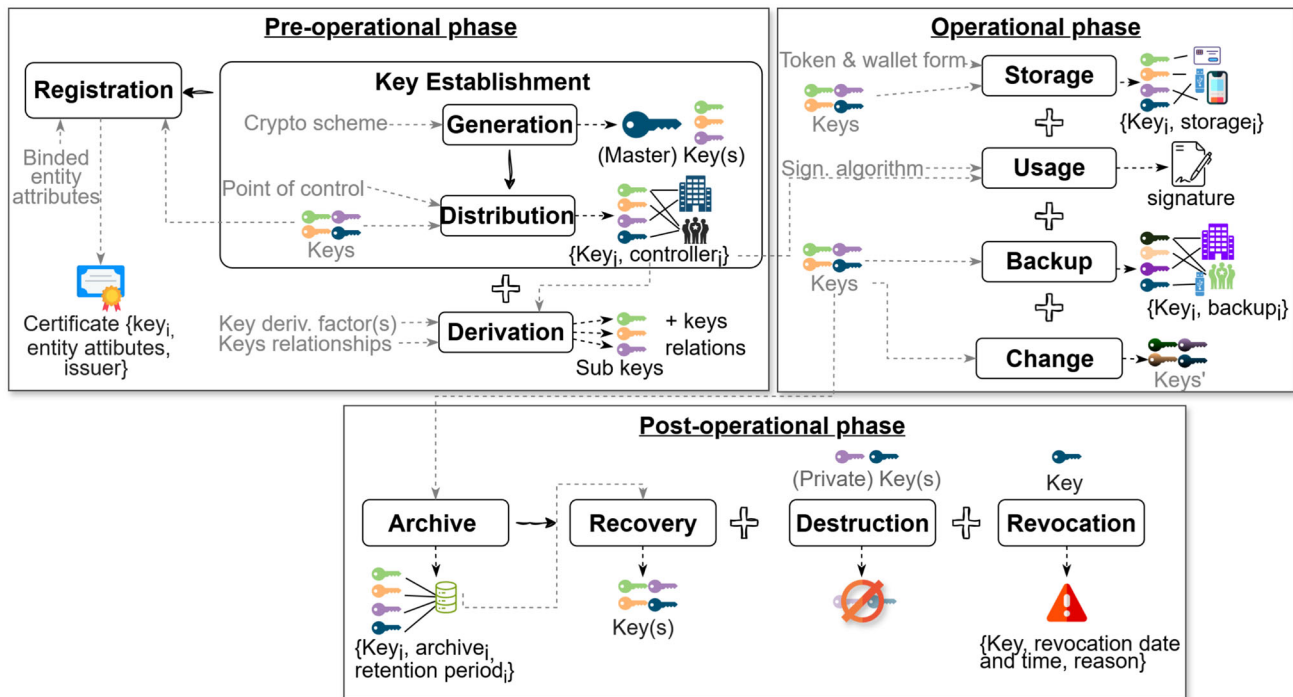


Fig. 1 Key management phases: stages, parameters and artefacts

be registered. Information provided during registration typically includes the identifier of the entity associated with the keying material (the owner) and the intended use of the keying material (i.e., a signing key) [14]. The registration should be done by a trusted Certification Authority (CA) that issues a certificate that binds the keys with the responsible entity's attributes.

The CAs are usually a separate organisation that provides key registration. The CAs may allow master key owners to certify other keys themselves, thus such a key owner starts acting as an intermediate certification authority (intermediate-CA).

However, caution shall be exercised because even a compromise of a single intermediate-CA could undermine the system's integrity. To decrease the risks, the chain of certificates to the root certificate, the scope and subject of intermediate-CA issued certificates shall be limited, or at least closely monitored. Otherwise, a compromised intermediate-CA might be used to issue certificates across organisational boundaries.

Sub-keys Derivation Data can be signed either directly by the master private key or by sub-keys (derived from the master key). For example, Bitcoin Improvement Proposal No. 32 (BIP32) [56] defines deterministic key derivation, where sub-keys are obtained by a *path* from the master key. The path is also needed to derive the correct public key for the signature verification algorithm. The deterministic derivation is spec-

ified only for certain schemes based on elliptic curves (as in Bitcoin) or lattices [3].

Another option for the derivation of sub-keys is to use a specialised share generation scheme which allows generating shares based on the "master (private) key" in order to use the shares for creating a threshold signature. The threshold signatures generally differ by the following parameters: (i) interactive or non-interactive (depending on whether the protocol requires all the shares to be generated by parties synchronously or not); (ii) following the selected signature scheme (e.g., RSA, ECDSA).

2.2.2 Operational Key Phase

Key storage The keys are stored on tokens that generate, protect and manage them. The token can be a software application installed on a general device, e.g., the one that requests the signature, or a full-fledged external device such as a Hardware Security Module (HSM). The dedicated device can be a USB token, a smartcard (e.g., JavaCard) or a trusted platform module built into an end-consumer device. Another device relevant for storing keys is a hardware wallet. Finally, deterministically derived keys do not require storage, as they can be generated on the spot – needing at least the threshold number of factors in the case of DKGs. The form of a token affects ODI's portability and usability.

Key usage The key usage is split into data signing and signature verification algorithms. The traditional, single-party,

signing algorithms allow a signer, with access to the (master) key, to digitally sign any data. Another option is threshold signature schemes, where at least a threshold of K out of all N signers need to collaborate to form a valid signature. In threshold signature schemes, each signer is responsible for their own key share. As a result, an attacker would need to compromise at least K signers to forge a valid signature.

Threshold signature protocols. Depending on the selected signing threshold protocol, a signature may be created through a different number of communication rounds between contributing parties (typically, from one to five [5, 47]). For example, the protocols in [6] and [44] require 5 rounds of communication. FROST [47] and protocols in [1, 19, 21] describe the one-round procedures for threshold signature creation enabled by pre-processing.

Some digital signature schemes, e.g., ECDSA or Schnorr, rely on the usage of a nonce. A *nonce* is a pseudo-random value that makes the signature scheme non-interactive. The nonces must not be reused to prevent private key recovery attacks. While a single party created signatures rely on a single place-generated nonce, a *collaboration nonce* is a nonce that is not generated by a single entity but is instead created in a distributed process by a threshold number of participants. Each participant contributes a secret random “share” of the nonce, and these shares are mathematically combined to produce the final, valid nonce. One round threshold signature is based on a pre-processing stage (i.e., “offline round”) for agreement between contributing parties on a collaboration nonce independently of a message to be signed. The creation of such a collaboration nonce in advance for having the message allows for speeding up the active signing phase (i.e., “online round”). Meanwhile, the provided level of security and the existing trust assumptions affect the required number of rounds.

Policy enforcement and accountability. If the master key is derived, the appropriate number of valid factors must be provided to derive the signing key. If BIP32 deterministic derivation is used, the path from the master key to the child key is also needed. Threshold schemes involving multiple signers improve recoverability and, thus, resilience – any quorum of more than the threshold of signers can create a signature using their key shares. Replacing a single ODI controller with a threshold group of K -out-of- N protects against privilege escalation. Most threshold signatures do not disclose which signers have participated in the signing. Thus, threshold signatures may help hide signers’ identities from the verifier. However, all signers are accountable for N -out-of- N threshold groups. Also, elaborate policies, like signing only during a specific time range (such as working hours), can be enforced through automated signer applications. Threshold groups can also be nested – e.g., top-level

3-out-of-3 group with two shares residing with individual representatives and the last share being an employee auto-signer that signs if another K -out-of- N employee group also provides a signature. Thereby, the ODI owner can trace back to which of the key shareholders participated in the signature creation or mandate policies.

Key backup To enable recovery of lost keys, an identity holder should back up the secret key elsewhere than the storage used during regular key operations. Such backup storage can be an additional HSM, a trusted execution environment for key backup [64] or a third-party custody of the keys. The latter carries the risks of identity theft through privilege escalation [64]. Therefore, in the case of a threshold signature, the users may opt out of backing up the original master key and, instead, require each key share custodian to back up their key shares. In this case, the backed-up key shares should not be stored in the same archive or backup storage to avoid the possibility of compromising the whole threshold signature by a single attack on such a storage [46].

For enhanced security, an identity holder may employ multiparty computation and Shamir’s Secret Sharing, distributing key shares among multiple custodians [64]. For multiparty computation, keys should be generated with DKG, while secret sharing requires the distribution of later-derived parts of a single key. A defined minimum threshold of custodians must provide their key shares to the identity holder to recover the backed-up key [64].

Change To proactively strengthen security and reduce the risk of compromise, private keys should be regularly updated (‘refreshed’), preventing them from ‘ageing’ and becoming weaker targets [14]. This involves replacing old keys with new ones on a scheduled or event-driven basis. A traditional update is done through a key rotation mechanism, which requires the update of both private and public keys. In the case of a threshold signature, the private key shares can be updated without the update of the common public key, which refers to the so-called *proactive refresh* (or proactive secret sharing).

At regular intervals, for instance, on a monthly basis (each such interval is called an ‘epoch’), all N participating entities will execute a proactive refresh protocol. The purpose of this protocol is to update their individual secret key shares without altering the corresponding public key. A critical security guarantee of this process is that an adversary, even if they manage to compromise fewer than K parties within any given epoch, will be unable to forge digital signatures. This holds true even if, over an extended period, the adversary cumulatively compromises all N parties by corrupting different subsets in successive epochs. Various subsequent research efforts have led to the development of specific proactive refresh protocols tailored for different threshold

cryptographic systems [4, 13, 18, 19, 28–30, 37, 38, 42, 48, 58].

2.2.3 Post-operational Key Phase

Archive and recovery A key archive plays the role of a repository for cryptographic keys and their associated metadata, facilitating recovery beyond the keys' cryptoperiod (the specific time frame during which a cryptographic key is authorised for use). Archiving is not universally required for all keys. In instances where key recoverability is mandated, such as following the expiration of its cryptoperiod, either the key must be archived, or the system must be designed to enable reconstruction, or re-derivation, of the key from archived data. Key retrieval, whether from archive storage or through reconstruction, is designated as key recovery. A trusted entity, such as an organisation affiliated with the key or a third-party service provider, must maintain the archive [14]. Depending on the organisation's security plan, the trusted entity should ensure different levels of security for the archive. Meanwhile, in general, the trusted entity should ensure (i) confidentiality for the archive through the access-control to the archived data; (ii) integrity through encryption of the archive or its data and, if needed, adding timestamp to the stored archive data, and (iii) availability through storing metadata about the key (and the reconstruction group, if applicable) in multiple separate locations [14].

Destruction All copies of private keys must be eradicated upon cessation of their operational requirements, including archival or reconstruction activities, to minimise compromise vulnerabilities. Secret and private keys shall be subjected to destruction procedures that eliminate all residual traces, precluding recovery by either physical or electronic means. Public keys may be retained or destroyed at the discretion of the key owner [14]. The destruction procedure should ensure that the stored keys cannot be restored with any physical or electronic means, e.g., by using the concept of *erasable memory* [24].

Revocation Revocation may be initiated for administrative purposes, such as when the key's original owner departs the organisation or a device containing the key is decommissioned. Alternatively, revocation may be executed on an emergency basis if there is evidence suggesting unauthorised disclosure or access. In either scenario, cryptographic keys should be revoked expeditiously following the determination of the revocation necessity. Notification of key revocation is necessary for entities that have used, are currently using, or would potentially use the revoked key, such as relying parties.

In the event of symmetric key revocation, all entities sharing the key must be notified, typically via a Compromised Key List (CKL). For asymmetric key pairs, revocation per-

tains to the private key. However, when public key certificates are deployed, the certificate containing the corresponding public key is revoked. Notification to relying parties is then disseminated through mechanisms such as Certificate Revocation Lists (CRLs) or the Online Certificate Status Protocol (OCSP) [14].

Notification may be effected through proactive dissemination to all potentially affected entities or via a mechanism allowing entities to query the key's status. The notification must include complete identification of the key, excluding the key material itself, the date and time of revocation, and, where applicable, the reason for revocation, such as key compromise [14].

To sum up, we identified mechanisms that help secure keys and signatures and, thus, ODI itself. The commonly used approaches of key management can address the problem of centralisation or bring zero trust in a targeted manner, while none of them addresses the problem on their own. Table 1 maps the reviewed mechanisms to the system characteristics they affect, and serves as the basis for the design decisions in Section 3. The mapping highlights the key phase and the mechanisms, the implementation of which primarily affects the respective system characteristics. An empty cell in the table refers to no direct effect or the lack of evidence which would confirm the impact.

3 Design of a Distributed Key Management System

In this section, we present a design of a Distributed Key Management System (DKMS) for ODI as depicted in Fig. 2. It enables zero trust within the organisation's information system and is compatible with any trust model. The design assumes that the identity's credentials rely on PKI and, thus, prescribes a selection of cryptographic and business mechanisms for each stage of the key lifecycle. The design aims to eliminate the centralisation of control over ODI throughout the lifetime of keys. It increases the set of representatives or custodians to enable partial custody of ODI.

To bring zero trust into ODI management, we propose a combination of the reviewed self-custodian and custodian control, namely, *partial custody*. An IS with centralised control over ODI is prone to a single point of failure [57] due to a strong assumption on a single fully trusted entity for signing messages on behalf of an organisation. If the entity is compromised or its administrator becomes malicious, the IS cannot assure the authenticity and, thus, integrity, of the provided information. The partial custody brings decentralisation of control and storage of keys for signing or recovery. Ergo, the system can operate with reduced trust in each sin-

Table 1 Mapping of system characteristics and mechanisms targeting them (reproduced from [8])

System characteristic	Pre operational phase				Operational phase				X-Road		
	Key gener.	Key derivation		Key distribution	Key storage		Key usage	Related targeted goals (Sect. 4.3)			
		Crypto scheme	Factors for deriv.		key	Enabled sign.			MPC	Point of control	Token form
Decentralisation											G1
Trustlessness						+					G1
Portability						+			+		G1
Multiple users		+									G1,G2,G5
Traceability											G2
Usability										+	G4
Prevent. priv. misuse	+									+	G3

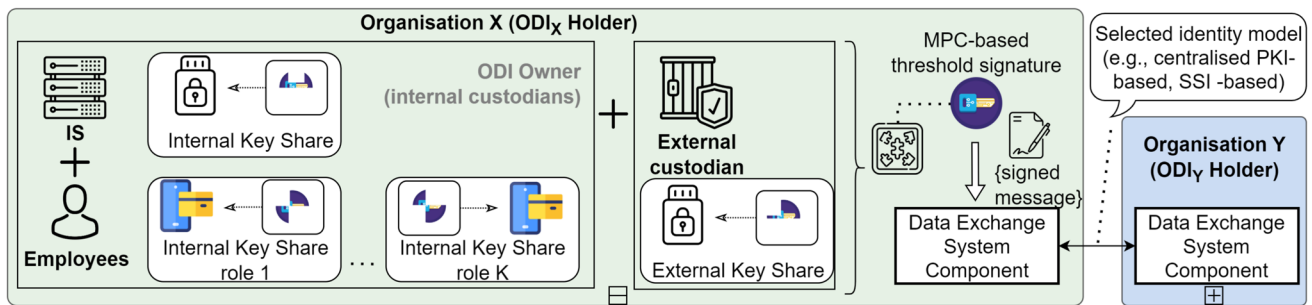


Fig. 2 DKMS for organisational digital identity in cross-organisational data exchange (reproduced from [8])

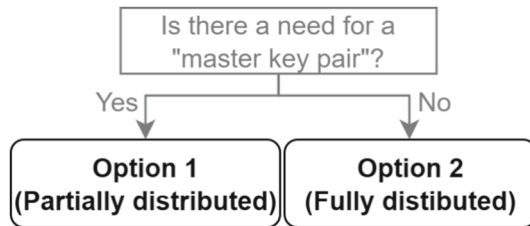


Fig. 3 Available key management options for the proposed DKMS

gle entity. The distribution of the keys eliminates the single point of failure since multiple key controllers would need to cooperate to compromise the system's security. By that, partial custody boosts the overall security posture of an IS and mitigates the risks associated with centralised control of keys. DKMS allows trustlessness, improves recoverability and resilience, supports traceability, and may enable the prevention of privilege misuse.

Based on the key management mechanisms reviewed in Section 2.2, in this section, we describe the components of the proposed DKMS for organisational digital identity in cross-organisational data exchange. The decision tree in Fig. 3 visualises the implementation choices for the proposed DKMS, which an organisation can make. The tree begins with the primary, mutually exclusive decision between Option 1 (Partially distributed) and Option 2 (Fully distributed), which defines the overarching approach for the entire key management lifecycle, depending on the need of having a single 'master key pair'. Thereby, the tree shows how these two sets of decisions combine to form two distinct and valid implementation paths for the DKMS. The following subsections elaborate the design choices available at each phase for both options.

Table 2 describes the options an organisation should make decisions about in order to implement the proposed DKMS. The options are provided for each phase and operation of the key management lifecycle that they concern.

Design approach. The DKMS does not introduce new cryptographic primitives, nor is this its goal. Instead, the

contribution lies in the system design layer: identifying, composing, and specifying the lifecycle management of existing threshold signature schemes that are well-established in the cryptographic literature but have not yet seen wide adoption in information systems practice. The design demonstrates how IS engineering can benefit from these existing primitives to enforce zero-trust policies for organisational digital identity.

3.1 Pre-operational Key Phase

To achieve zero trust through partial custody in DKMS, the decentralisation of control over keys may be introduced in one of two steps – during key derivation or generation (Option 1 and Option 2 in Table 2 respectively).

Key generation and derivation First, the keys can be initially generated in a decentralised manner. This refers to a distributed key generation (DKG) mechanism. In this way, an organisation selects a DKG for the selected signature scheme that results in a set of key shares generated by key share custodians. This set of shares together can be used to produce a master key, yet there is no single point in time when a (master) private key exists in one place.

Key distribution Second, if there should be one centrally generated master key, decentralisation can be introduced in the step of key derivation and distribution using key shares generation based on the existing 'master key pair' by a trusted dealer. Depending on the need of having key share custodians to interact with each other during the shares generation, the derivation protocol may be either interactive or non-interactive. Finally, the organisation should select such a key shares derivation protocol so that it corresponds to the final used signature scheme (e.g., RSA or ECDSA).

Regardless of the phase during which key shares are generated and distributed to semi-trusted custodians, an organisation can derive multiple keys to enable distributed key storage and threshold signature creation. Semi-trusted custodians can be internal entities (IS components or employees) and external service providers (i.e. external custodians).

Table 2 Description of DKMS implementation options for each key management phase

KM phase	KM function	Options	
		Option 1 (Partially distributed)	Option 2 (Fully distributed)
Pre-operational	Generation	Use a key generation protocol for the selected signature scheme that produces 'master key' pair	Use a distributed key generation (DKG) protocol for the selected signature scheme that produces a master public key and corresponding private key shares (DKG is executed by key share custodians)
	Derivation	Use a share generation scheme based on the 'master (private) key' and selecting the following parameters: (i) interactive or non-interactive; (ii) following the selected signature scheme (e.g., RSA, ECDSA)	(No derivation is needed)
	Distribution	A trusted dealer distributes the derived shares directly to the custodians	(No distribution is needed)
Operational	Storage	Each custodian uses a selected token form for its shar (software or hardware tokens)	
	Usage	The custodians follow the selected signing and verification protocol for the signature scheme w.r.t. the signature protocol used for the initial keys generation. The signature protocol (both for signing and verification) is defined by the following parameters based on the expected security level and assumptions: (i) number of communication rounds (typically from 1 to 5); (ii) synchronous or asynchronous; (iii) accountable or non-accountable	
	Backup	(i) A trusted dealer backs up the shares of the 'master private key' generated using Shamir Secret Sharing; (ii) Each custodian backs up its share as it is in the selected storage, and no backup is created for a 'master key'.	(i) Each custodian creates the sub-share of its shares using Shamir Secret Sharing and backs up these sub-shares in different storage. (ii) Each custodian backs up its share as it is in the selected storage
	Change	Proactive refresh based on the selected threshold signature	
Post-operational	Archive	A trusted dealer archives the secret shares of the 'master private key' which are generated with Shamir Secret Sharing in the encrypted form to be accessible for recovery after cryptoperiod	Custodians use proactive secret sharing for key share recovery protocol
	Recovery	(not recommended for private key)	
	Destruction	Use the erasable memory	
	Revocation	Conduct revocation of the certificate for the master public key and the respective threshold signature	

When new employees come, and old ones leave, the distributed key generation re-sharing can be performed while keeping the (certified) public key the same.

For the point of control during keys' distribution, we use *partial custody* achieved through *threshold signatures*, where internal (e.g., employees' roles and an information system component) and external custodians are the contributors to the signing of messages on behalf of an ODI.

3.2 Operational Key Phase

Key storage Users' keys are the key shares for the threshold signature. Access to these shares can be based on the user's identity, following internal organisation policy. Each entity can use *different token forms* to store its share. Thereby, the proposed DKMS has threshold signatures as a policy decision point component in respect of zero trust architecture [60].

Key usage For the keys usage, an organisation should select a threshold variant of the signing protocol for their preferred signature scheme (e.g., RSA or ECDSA) based on the expected security level and assumptions. If the design is applied to the already existing data exchange system, the selection of a signature scheme also depends on what set of schemes is supported by the targeted data exchange system. Specifically, the selected threshold protocols can be: (i) consisting of various number of communication rounds between key share custodians, both during distributed key generation and actual message signing; (ii) synchronous or asynchronous; (iii) accountable or non-accountable [1, 5, 19, 21, 44, 47]. The synchronous protocols assume that network payloads have a known, fixed maximum delay. Protocols designed under this assumption can operate in well-defined rounds, guaranteeing that all messages sent in one round will be signed before the next signature should be created [15]. Meanwhile, the asynchronous protocols are more flexible, allowing for arbitrary and unpredictable delays. The only guarantee is that a message will eventually be signed [15]. In the accountable threshold signing protocols, the signature completely identifies the set of signers [16]. Therefore, an organisation's representative can identify which of the custodians contributed to the creation of the threshold signature on behalf of ODI. In contrast, non-accountable (or fully private) threshold signature schemes reveal nothing about the set of signers that generated the signature [16].

A threshold group may include signers of different automation levels: fully automated signers, human signers, and *semi-automated signers*. A *fully automated signer* is an IS component that applies a predefined, machine-enforceable policy, such as restricting signing to a specified time window or verifying the requester's IP address, and contributes its partial signature without human involvement. A *human signer* holds a key share on a personal token (e.g., a smart-card) and actively approves each signing request, providing the strongest form of human oversight at the cost of latency.

A *semi-automated signer* is an IS component that acts on behalf of a designated human representative, but conditions its participation on evidence of that representative's recent authentication. Specifically, the semi-automated signer contributes its partial signature automatically only if the associated human has authenticated within a configurable time

window (an *activity epoch*, e.g., within the last hour or since the start of the working day). If no recent authentication is recorded, the signer withholds its share, blocking signature creation until the human re-authenticates.

This design allows an organisation to cryptographically enforce a human-in-the-loop signing policy ensuring that a live, authenticated person is accountable for signing activity. Meanwhile, this allows avoiding the latency of requiring that person to approve every individual message in real time. The semi-automated signer is particularly useful when a human participant in the threshold group is the *responder* rather than the *initiator* of a message, i.e., the human is not actively present at the moment of signing, yet the policy still mandates their involvement. The length of the activity epoch is a configurable parameter that the organisation sets according to its security policy balancing the strength of the human-in-the-loop guarantee against operational convenience.

Key backup When a "master private key" is split into multiple shares, two primary backup strategies emerge, each with its own trade-offs. The first involves a centralised approach where a trusted dealer uses Shamir Secret Sharing (SSS) and creates new key shares from the "master private key". Afterwards, such SSS-based key shares are backed up. While this simplifies the recovery process and centralises control over backup, it reintroduces a critical single point of failure that negates the core security benefit of the secret sharing scheme. The second and more secure approach is decentralised, where each share custodian is individually responsible for backing up their own unique share. This strategy eliminates a single point of failure and significantly increases the effort an attacker would need to compromise the key, but it complicates the recovery process due to the need for coordination and potential inconsistencies in individual backup practices. In the case of the fully distributed option (Option 2), each share custodian is individually responsible for backing up their own unique share. Thus, each custodian can back up its share as it is, or each custodian can additionally split its share into sub-shares using SSS and store these sub-shares.

Change For updating the keys within their operational phase, a proactive refresh of the shares should be executed by the custodians for any of the selected earlier implementation options. The specific protocol of refresh should be selected w.r.t. the selected threshold mechanism and signature, e.g., from the ones presented in the following works [4, 13, 18, 19, 28–30, 37, 38, 42, 48, 58].

3.3 Post-operational Key Phase

Archive and recovery For Option 1, the custodians may archive the secret shares of the "master private key" generated with Shamir Secret Sharing in the encrypted form to be

accessible for recovery after the crypto-period. However, we do not recommend doing any archiving for the master private key. If Option 2 is selected, the share recovery protocol as a part of proactive secret sharing, which is described in [39], may be used for recovering shares. Besides, the existence of a group and some identification of group members must also be added to the archive to allow recovery initiation.

Destruction and revocation When the organisational key and its shares are no longer needed, all copies and backups must be permanently destroyed to prevent potential security compromises. Therefore, at the operational phase, the custodians should ensure the usage of erasable memory for the key share to allow for destruction. A key should be revoked immediately if it is compromised or no longer in use. Once a key is revoked, all parties that relied on it must be notified, as the threshold signature created after the main key revocation will not be valid anymore.

3.4 DKMS design summary

This subsection summarises the security properties that the proposed DKMS achieves through the combination of mechanisms described above, and states the conditions under which the system should be applied. In this paper, we show that a threshold signature allows organisations to distribute control over the identity. First, key shares allow the distribution of trust among the organisational parties – employees and IS components – which are primary representatives of the organisation and the initiators of messages. Second, it helps maintain access control in case of employee turnover in an organisation. Finally, a threshold of K shares protects the ODI and organisation against a compromise of up to $K - 1$ signing parties.

Additionally, DKMS brings access policy enforcement. In a centralised trust model, the external key controllers need to be trusted with complying with the access policies. Meanwhile, the access of internal actors to the data requests on behalf of the organisation is controlled by the (role-based or attribute-based) access control systems or the general rule-based policies. With threshold signatures, the organisation can implement these access policies by assigning shares to individual internal IS components that are then responsible for enforcing the policies through cryptographic protocols. For example, an IS can include a component that signs only at a specified time range and another that verifies the requester's IP address. Compliance with the access policy is enforceable by the organisation and does not require any changes on the verifying side.

The example DKMS group composition described above illustrates these principles in a concrete setting. We stress that including human-in-the-loop is optional, and individual signers can be fully-automated processes.

The proposed system should be applied when (i) organisational identity relies on public key infrastructure; (ii) organisation uses digital identity for cross-organisational IS (e.g., through data exchange systems for connecting standalone ISs); (iii) organisation wants to remove a single point of control over its identity; (iv) multiple organisation representatives can contribute to the signing.

4 DKMS Design Evaluation

This section demonstrates and evaluates the proposed DKMS design through a proof-of-concept implementation for X-Road, a real-world data exchange system that satisfies all four applicability conditions stated above at the end of Section 3.

4.1 X-Road Use Case

X-Road is a distributed data exchange system between Information Systems (ISs) within the trusted network [54]. X-Road relies on PKI to ensure trust between members of the network. Members are organisations with their ISs used for operations by internal users (e.g., employees) and external users (e.g., customers). For cross-organisational data exchange, a component called *Security Server* (SS) serves as an intermediary [54]. Let us consider Member-Client and Member-Provider as X-Road Members with set-up SSs. The Client requests data from the Provider. The SSs are the components which are responsible for the PKI-verified message exchange. The Client's SS signs the data requests on behalf of the Client. The Provider's SS verifies the Client's signature in the received request and signs the response on behalf of the Provider. Finally, the response signature is verified by the Client's SS. Additionally, SS logs the data exchange so that a third party can check the Member's request or response signature if a proof is needed (e.g., in a court) [54]. In the current set-up, the X-Road is oriented toward protecting the confidentiality and integrity of the data exchange messages delivered via the public Internet. Meanwhile, all the entities inside the network are fully trusted (including SSs) based on the PKI certificates, and SS is not necessarily managed by the organisation that uses it.

Thus, we aim to eliminate the delegation of an ODI to a single SS component and enable the zero trust principle to manage identity and improve security. The following goals should be achieved: **G₁**. Member's ODI cryptomaterial is not in the sole control of one entity. **G₂**. Member can trace back the internal initiator of the message sent on Member's behalf through X-Road. **G₃**. Member can define the access of internal entities to operations on its behalf. **G₄**. The system is backwards compatible. **G₅**. No single entity can create a valid signature.

Table 1 maps X-Road characteristics (Sec. 2.1), which can be achieved using the reviewed key management mechanisms (Sec. 2.2).

To achieve our goals, we propose targeting specific system characteristics. For G_1 , we aim for decentralisation, trustlessness, portability, and support for multiple users. We focus on multiple-user support and traceability for G_2 and G_5 . For G_3 , we emphasise preventing privilege escalation. Altogether, the stated goals aim to protect the integrity of the data exchanges within the trusted network by assuring message authenticity and proof of origin.

4.2 System Design

We set up the distributed key management system in X-Road, as depicted in Fig. 4, to evaluate its feasibility. White classes depict current X-Road entities, green classes represent the added DKMS components, and dark grey represents services used to implement the DKMS for X-Road. Currently, each Member has a fully trusted custodian, represented by a Security Server (SS). SS is responsible for ODI's key management and message signing on the organisation's behalf. The key can be either managed by an external service provider or by the Member itself. To eliminate the centralised management, we use threshold signatures to increase the number of key controllers from one to N , where at least $K \leq N$ controllers are needed to create a valid signature. Each controller can validate different policy rules for using its key.

In the X-Road use-case, we propose to share the Member's identity key material between the three key share controllers which already participate in the process of message exchange: (i) Member's information system used for creating a message; (ii) Client's employee making a data request so to ensure that the request comes from Clients' identity; (iii) Security Server (SS) used by a Member for the message exchange.

The selection of the threshold signature scheme is constrained by the signature schemes supported by X-Road, and affects both the key generation process and signing performance. For multiparty threshold signatures, we use an open-source threshold signing platform. If employees are involved in the signing group, they may store their key share on a cryptographic smartcard, while the external custodial (represented by an SS) and Member's IS may use Hardware Security Modules (HSMs) as key share storage. The threshold variants of both RSA and ECDSA signature schemes [32, 63] ensure full backwards compatibility with the latest X-Road version 7.7.0.

Threshold ECDSA is supported since the version 7.6.0, threshold RSA is supported even in older version. The selected RSA scheme requires a trusted dealer for generating and dealing with the individual key shares [63]. This limitation could be overcome by using the updated threshold

RSA variant [22]. The selected threshold ECDSA protocol includes distributed key generation. For RSA, we chose the Member's appointed representative (e.g., administrator) to be a dealer for ODI keys; as such, it has control over who the individual shares are available to. After the shares are generated, the dealer must securely delete the private parameters used during the generation of the shares. In case of ECDSA, the dedicated signers perform the distributed key generation.

Fig. 4 depicts the design of the DKMS for the Client's side only. Such implementation assures the integrity of data requests sent on behalf of the Client's ODI. The analogous DKMS can also be implemented at the Provider to ensure the authenticity of the response to the Client's request.

Threshold signatures increase the security of a private key, but the particular deployment setting affects the signing time performance. Depending on the (geo-)distribution of the shares, the network delay could slow down the signature creation. One mitigation is to prefer the signers which are available at the moment of signature creation. Additionally, the threshold variants of signature schemes, such as RSA or ECDSA, differ in the number of communication rounds, size of messages and computation complexity, all of which affect the time of signature creation. We discuss the concrete values for the performed experiments later in the Section 4.3.4.

4.3 DKMS for X-Road Implementation

In the DKMS proof of concept implementation, we connect the threshold signing platform to X-Road to achieve the proposed design through the supported PKCS #11 cryptographic interface. The chosen threshold signature schemes, Rivest-Shamir-Adleman (RSA) and Elliptic Curve Digital Signature Algorithm (ECDSA), are compatible with the ones used in X-Road. The threshold signing platform includes a coordination server and clients (signing service). The threshold signing server can be deployed either by the Member or by its SS, and the clients must be accessible to the signing parties. The repository with the proof of concept implementation and deployment manual is available at [34].

To evaluate the quality of the developed proof of concept, first, we conduct a qualitative assessment to check the meeting of the targeted goals and system characteristics. Furthermore, we do the quantitative assessment of the time for message signing and transfer, compared to other common signing tokens.

4.3.1 Testing scenario

We consider the following two-member X-Road scenario to validate that the proposed DKMS achieves the goals stated in Sec. 4.1. One Member is a governmental data repository

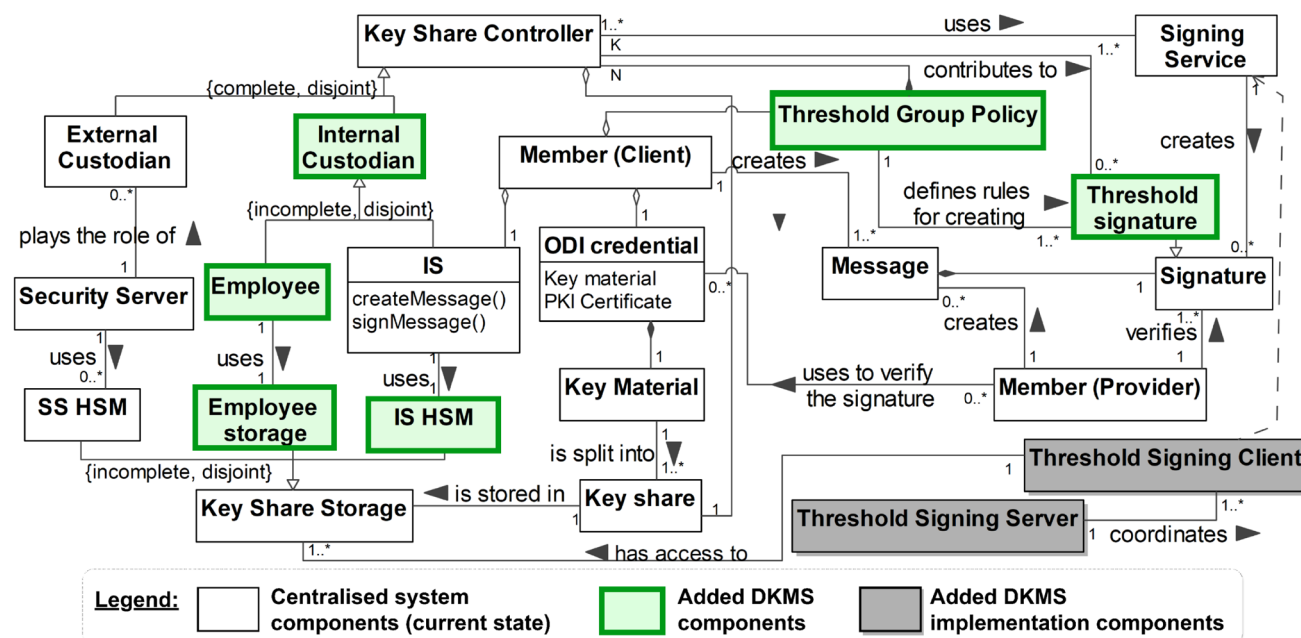


Fig. 4 DKMS for organisational digital identity in X-Road (N – the size of a group, K – threshold number of controllers contributing to a signature, $K \leq N$; reproduced from [8]).

provider (e.g., a health agency), and another is a hospital. The hospital (Client) has an information system IS_C . The client uses Security Server SS_C for the data exchange in X-Road. The health board (Provider) has an information system IS_P that stores health-related data. Provider uses Security Server SS_P for the data exchange. IS_C has a user interface for internal usage by three roles: employees who can be doctors, receptionists, and interns who are practising students from the medical school. Interns work in the hospital for a short time during their practice time and leave the hospital afterwards. Interns only have a temporary need to use the system.

4.3.2 Setup

Let us assume the setup where the Client's ODI key material should be distributed between IS_C , SS_C , and employees' roles. Thus, there should be at least five shares, where each role representative holds the same key share. To allow excluding some of the key shares, we propose to derive more than one key share for interns so that after their practice time passes, the interns' key share is defined as "deprecated" (signing service does not allow the use of it anymore) and other pre-generated key shares are made available to the next round of new interns. A minimum threshold of three is chosen to require the simultaneous involvement of the IS component, the SS, and at least one employee role, directly enforcing the

zero-trust policy. In particular, we have chosen a 3-out-of-5 threshold signing group³.

Fig. 5 depicts the process of distributing shares among three custodians. Fig. 6 depicts a process of message signing using DKMS in X-Road with a 3-out-of-5 threshold signature.

The described scenario with a 3-out-of-5 threshold group assumes having a human, who is online and is able to sign messages. In the case of the message which contains a data request, the human is the message initiator and is indeed already online and is able to sign the message without a delay. However, if the data provider implements an analogous DKMS system for approving the sending of requested data, a data provider's employee does not initiate such messages themselves and instead they are only appointed as verifiers. Such human involvement, as a result, leads to higher latency. To mitigate the latency associated with human involvement, the DKMS system design allows for the inclusion of a semi-automated signer within the 3-out-of-5 threshold group. Such a semi-automated signer can act on behalf of a human representative who, although physically present and authenticated (for example, at the start of the workday or within the last hour), may not be actively participating in the real-time signing procedure. The semi-automated signer is configured to sign a message only if a human from the same role has recently authenticated. This mechanism ensures that the organisational signing policy –

³ The other tested configurations are shown in Table 3.

Fig. 5 A process of distributing shares among three custodians for DKMS in X-Road using Option 1 [7].

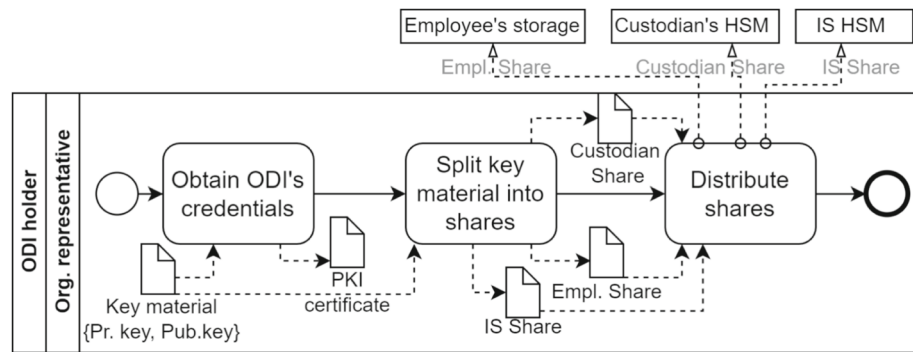
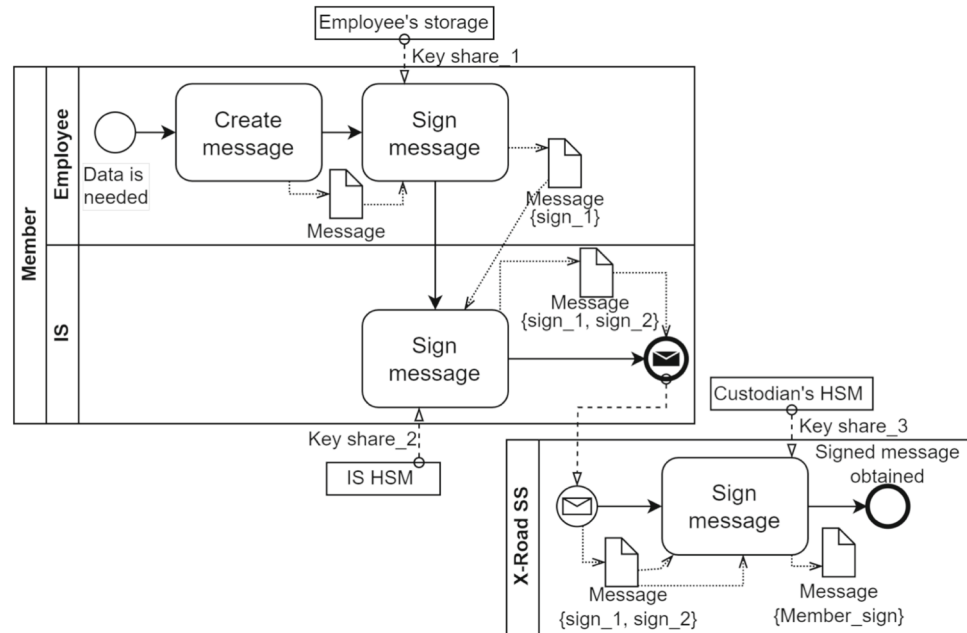


Fig. 6 A process of message signing using DKMS in X-Road with a 3-out-of-5 threshold signature [7].



which requires a human-in-the-loop – is cryptographically enforced even when the human representative is temporarily offline. It maintains security by linking automated signing to a recent human authentication, while simultaneously improving the efficiency of the overall system.

4.3.3 Goals achievement

During the operational phase, the keys used to create a signature on behalf of ODI are distributed among at least three entities – SS, IS, and some employees (G_1). The setup threshold group policy used by the signing service guarantees that key share controllers from the threshold group all participate in the signature creation. Thus, Members can partially trace back the entities (to the level of roles) who initiated the message (G_2). An ODI representative who defines a threshold group policy can define among which internal entities key shares are distributed, how the shares are derived, and how to revoke access. Thus, the signing service using the threshold group policy controls that only the active key shares can

be used for the signature creation (G_3). The implementation uses an RSA and ECDSA-based threshold signature signing scheme that is backwards compatible with the standard RSA and ECDSA verification algorithm (G_4). The setup threshold group policy used by a signing service guarantees that IS, SS and employee contributed to the signature creation (G_5). As a result, the proposed DKMS for X-Road allows us to satisfy the stated goals and enables distribution control over Members' ODIs during the operational phase of keys.

4.3.4 System assessment

To assess the feasibility of the DKMS usage within the X-Road use case, we measure two parameters: (i) the payload size of the additional communication introduced by the threshold signing protocol, which affects network overhead, and (ii) the round trip time (RTT) of a full Client-Provider data exchange, which reflects the end-to-end latency experienced by the system's users. Each measurement is conducted for two different signature schemes (i.e., RSA and ECDSA)

with varying threshold group size setups (2-of-2, 3-of-5, and 11-of-15).

Payload size. For the key generation, the chosen threshold RSA [63] scheme utilises a trusted dealer and requires two rounds of communication with the signers. Assuming 2048-bit RSA keys, the first communication round contains only a few bytes encoding the group parameters. The second round, which distributes the actual shares, is around 1kB for each signing party.

The signing communication also requires two rounds. During this communication, the partial signature data (hereafter referred to as “payload”) is exchanged between threshold signers. The payload size of the first round contains mainly the hash of the to-be-signed message, and is as large as several tens of bytes. During the second round, the partially signed message together with a proof of its correctness is communicated, amounting to roughly 1.5kB per signer.

For the chosen threshold ECDSA [33], the distributed key generation has six communication rounds. For the three tested threshold settings (see Table 3), the payload sizes range between 100B–7.5kB (2-of-2), 400B–30kB (3-of-5), and 1300B–105kB (11-of-15). There are ten communication rounds for signing, with payload size ranging between 100B–3.5kB (2-of-2), 150B–7kB (3-of-5), and 760B–35kB (11-of-15).

Round trip time. For time measurements, the previously described testing scenario with Client’s IS_C , SS_C and Provider’s IS_P , SS_P is used. All the components are running on a single laptop device, with four 8GB LPDDR5 RAM sticks and AMD Ryzen 7 PRO 7840U CPU, as Docker containers.⁴ The Provider uses X-Road’s built-in software token (SoftToken) in SS_P , and the Client uses a range of signing solutions, as described later. The group’s coordination server is deployed to IS_C and connected through PKCS#11 interface to SS_C . The signing applications are individual running processes that automatically sign any incoming requests. This localised deployment allows us to measure the system’s throughput and compare various signing options with a negligible network delay. We target backwards-compatibility, but at the expense of implementation changes in X-Road, the request to SS_C could be partially signed to avoid extra network requests within the group.

The Round Trip Time (RTT) measurement starts with the IS_C requesting data from the IS_P and stops when the response is received. The mean and standard deviation of the RTT over 1000 measurements is in Table 3. The experiments are run

using the pre-established key shares and, therefore, the time for running the key generation protocol is not reflected in the experiment results. We run signing experiments for both RSA and ECDSA signature schemes and varied the signing tokens on the Client’s side. We compare software-only ones – the X-Road’s built-in SoftToken (ver. 7.7.0) and test hardware security module SoftHSM (ver. 2.6.1), the off-the-shelf hardware tokens – YubiKey (5C NFC) and Nuvoton Technologies Corporation TPM (ver. 7.2.3.1). Finally, to compare each single-party signing token, we run an experiment on the three selected threshold group setups – 2-out-of-2, 3-out-of-5, and 11-out-of-15.

The RTT results in Table 3 show that for RSA signature the RTT varies from 32ms to 267ms for different single-party signing tokens. The RTT for small 2-out-of-2 and 3-out-of-5 threshold groups is 191ms and 272ms, respectively. For ECDSA signature, RTT varies from 30ms to 190ms for different single-party signing tokens, while RTT for 2-out-of-2 and 3-out-of-5 threshold groups is 501ms and 677ms. For the larger threshold group of 11-out-of-15, the RTT is 705ms for the RSA signature scheme and 3081 ms for the ECDSA signature. While RTT is slower with increased group sizes, the results show the practicality of the threshold signatures when compared to the single-party signing tokens.

Let us take the RTT for single-party RSA tokens, which range from 32ms to 267ms, as the performance benchmark. The small 3-of-5 threshold group has an RTT of 272ms, which is in the same range as the slowest single-party hardware token. Thus, the experimental results suggest that for smaller threshold groups for RSA signatures, the performance penalty is manageable and could be acceptable as the cost for achieving zero trust. To assess the scalability of the RSA threshold signature, we increase the group to 11-of-15, where the RTT rises to 705ms. While this is a noticeable increase (about 2.6 times more than the 3-of-5 setup), it remains under one second, which might still be viable for many use cases.

The results show that while single-party ECDSA can be faster than RSA, but its performance in larger threshold signature setups is worse. A small 3-of-5 group ECDSA has an RTT of 677ms, which is over 3.5 times slower than the slowest single-party ECDSA token and over 2.4 times slower than the equivalent RSA threshold setup. The RTT for the large 11-out-of-15 threshold group is more than 4 times higher for ECDSA than for RSA (3081ms vs 705ms), but both cases still practically usable, especially when at least one signing party is a human.

The main limitation of the presented implementation of DKMS for X-Road is relying on a trusted central party (dealer) when the RSA scheme is used during the pre-

⁴ We used X-Road in version 7.7.0 and deployed the containers according to the instructions at <https://github.com/nordic-institute/X-Road/tree/develop/Docker/xrd-dev-stack>.

Table 3 Round Trip Time (RTT)* comparison of the Client-Provider** data exchange

Scheme	Single-Party Signing Token		YubiKey 5C NFC	TPM NTC 7.2.3.1	Threshold Group Setup		
	SoftToken	SoftHSM			2-of-2	3-of-5	11-of-15
RSA	32 ms (± 5)	36 ms (± 4)	174 ms (± 8)	267 ms (± 11)	191 ms (± 35)	272 ms (± 36)	705 ms (± 80)
ECDSA	32 ms (± 5)	30 ms (± 5)	100 ms (± 5)	190 ms (± 6)	501 ms (± 50)	677 ms (± 48)	3081 ms (± 345)

* The RTT mean and standard deviation (in brackets) are calculated across 1000 requests and reported in milliseconds.

** The Provider is set up with X-Road's built-in SoftToken, and the Client's signing token varies from single-party ones to different threshold groups

operational phase. Thus, our system design assumes trust in an organisation representative responsible for key generation, certification and key shares distribution. Hence, in case of RSA, the presented proof of concept is prone to a single point of failure in the pre-operational key phase when the identity's keys are generated, and the threshold group policy and threshold signing service are not enabled. This limitation could be overcome by using the updated threshold RSA variant [22].

Final performance affecting consideration is whether the signers are fully automated, or there is a human-in-the-loop, or a combination of the two is used. We stress that the system can be deployed and function fully automatically, with each signer possibly verifying a different set of conditions, e.g., time, IP address, or being online within the specified time range. If desired, some of the signers can be operated by humans. Naturally, the data exchange would be delayed until the human issues the partial signature. Human oversight allows for much finer scrutiny of the requests prior to signing.

5 Related Work

For managing the keys related to ODI, multiple access control models can be used (e.g., role-based, attribute-based, and discretionary [61]). But to the best of our knowledge, none of the traditional access control models considers the control over keys in view of the zero trust paradigm and the context of cross-organisational data exchange. The closest to our research focus is the architecture vision for the data exchange platform Simpl [26]. There, the authors review three Identity Management (IdM) models: centralised based on PKI [26], hybrid based on PKI and extensions for verifiable credentials [26], and self-sovereign identity with a distributed ledger [2, 11, 26]. However, the three models differ in the procedure of trust establishment between the organisations, leaving out of scope users who act on behalf of an organisation. Identification of such physical entities is proposed to be handled as a separate task through proprietary identification solutions (e.g., Microsoft Single Sign-on, OpenID) [26], internal access control systems, or an employee's wallet (digital or smartcards). Therefore, in

this paper, the proposed DKMS bridges two IdM systems – (i) for end-users of organisational IS and ODI, and (ii) for ODIs and cross-organisational data exchange. As a result, the novelty of the proposed DKMS is in its ability to enforce organisational identity policies for the system users through threshold signature, performed in an opt-in and backwards-compatible way directly applicable to existing information systems. The solution cryptographically guarantees compliance with the policies and, thus, enables zero trust within the IS.

Similarly to this paper, the work in [53] proposes an approach for implementing a Zero Trust strategy in a collaborative system between organisations. Their model features a decentralised identity management system that uses decentralised identifiers (DiDs) and verifiable credentials (VCs) for organisations' employees (where the organisation is an identity provider) and a blockchain registry. While both work addresses securing organisational digital identity, [53] presents a broader Zero Trust framework for identity and access control without any implementation details. Our research, in contrast, provides a more specialised solution focused on the distributed custody and full-lifecycle management of cryptographic keys, allowing organisations to implement it based on the detailed design.

In [12], the authors propose a scheme for secure data exchange that integrates signature delegation with a distributed reputation management system. While the paper also advocates for delegating signature creation to multiple external custodians, they aim to remove signing responsibility from the identity owner but control the trustworthiness of custodians through reputation scores. Thereby, the paper uses a similar approach but has an almost opposite goal for the signature delegation.

The closest to the proposed DKMS design is the system proposed in [41]. The authors propose an identity-based mediated RSA (IB-mRSA) encryption system that addresses a similar core problem: securing a private key by distributing it to multiple parties. The key differences lie in their approach and objectives. IB-mRSA is focused on a two-party split between a user and an online Security Mediator (SEM). This

is primarily to enable fast and fine-grained revocation of signing privileges. It still relies on a single, trusted SEM. Thereby, IB-mRSA is an alternative approach to the DKMS that also addresses the risks of centralised keys, but with a different mechanism and for a different primary purpose. As a result, the DKMS is a more general and flexible design compared to IB-mRSA as (i) it eliminates the need for a single, trusted third-party mediator (the SEM); (ii) the DKM enables a more robust, N-of-M policy where any subset of a threshold group can sign, rather than a fixed two-party split. Additionally, in this paper, we provide a full-lifecycle solution, including key change, backup, archiving, and destruction, which is not a stated focus of the IB-mRSA scheme, making it harder to be implemented in the running information systems.

6 Limitation

The paper does not consider the legal implications of the proposed design. The proposed DKMS allows the usage of smartcards and personal devices as hardware security modules for key shares. Thus, signatures created by physical custodians can be legally binding in such a way. The reason for such a conclusion is that the same technology is used in Smart-ID⁵, an electronic authentication tool used for governmental services in Estonia. As Smart-ID is a qualified signature creation device, the generated signatures are legally binding. However, using a threshold $K < N$ eliminates the non-repudiation nature of digital signatures, which could have legal consequences.

The implementation of the proposed DKMS may vary – requiring human-in-the-loop or semi-automated – depending on the organisation’s internal policy. Waiting on the employee’s approval is a limitation on one hand, but complies with the four-eyes security principle on the other. While including human-in-the-loop means the need for employees’ active approval involvement in the ODI usage for the signature creation, such involvement may be conducted only for the selected type of messages (e.g., for the requests for sensitive data on behalf of ODI), allowing the signing of messages in batch (instead of each single message) or based on the employees’ availability.

Integrating the proposed DKMS adds overhead for the operator of the threshold signing platform and the individual clients’ signing application. Optionally, storing the signing shares on dedicated tokens, such as JavaCards, improves the security, but brings additional financial costs.

In our research, we leave the post-quantum cryptography (PQC) out of scope. However, the PQC-based threshold signatures can also be used in the DKMS design. While we do not have the experiment results for the PQC threshold usage

as a part of the DKMS, we expect them to introduce higher delays to the message exchange compared to the classical cryptography-based signature schemes.

The experimental evaluation was conducted in a controlled local environment, with all system components deployed as Docker containers on a single device. The reported RTT values therefore do not include network delays between geographically distributed signers, which would increase latency in real-world deployments, particularly for larger threshold groups. The results should thus be interpreted as a lower bound on the expected signing latency rather than a representation of production performance.

The RSA-based DKMS implementation relies on a trusted dealer for key share generation during the pre-operational phase, as noted in Section 4.3. This introduces a single point of failure in the pre-operational phase, which the current proof-of-concept does not eliminate. The dealer-free RSA variant [22] could address this, but has not been implemented or evaluated in this work.

7 Conclusion

This paper has investigated how organisations can secure their digital identity (ODI) from abuse by a centralised controller during cross-organisational data exchange. The theoretical contribution of the paper is a key management system for information systems which enables cryptographically assured access policies enforcement that brings zero trust to the control over organisational identity. The practical contribution includes proof-of-concept implementation of the proposed design for the X-Road data exchange system and the library for RSA-threshold implementation available in [35]. In this study, we have reached several insights.

The existing methods for managing organisational identity either assume complete trust in the selected internal or external entities for operating on behalf of the organisation [26, 61] or require extensive infrastructural changes to avoid centralisation if self-sovereign identity ecosystems [11, 26] are used. As an alternative, this paper proposes a Distributed Key Management System (DKMS) that allows partial custody by employing threshold signatures to distribute control over the identity between multiple semi-trusted entities. The system ensures the authenticity of the messages sent on behalf of an organisation and removes centralisation. The performance results show that threshold signatures are comparable to state-of-the-art key management solutions.

The implementation of DKMS demonstrates that the system design helps to achieve decentralisation and traceability of signed message origin, as well as the ability to involve multiple ODI users whose access rights can be controlled to prevent privilege escalation. Moreover, the proposed system is backwards-compatible without major changes for all the

⁵ <https://www.id.ee/en/article/smart-id/>

parties involved in the message exchange. As the system evaluation shows, the approach applies to both RSA and ECDSA, threshold variants of two different signature schemes.

Based on the experimental results, the performance of a DKMS solution depends on the chosen signature algorithm. An RSA-based DKMS is promising as the performance penalty for small threshold groups is manageable and comparable to single-party hardware tokens. Even when scaled to larger groups, its latency remains under one second, making it viable for many use cases. In contrast, an ECDSA-based DKMS introduces higher latency even in small groups, reaching over three seconds for the 11-of-15 group. However, such delays may still be acceptable in scenarios where at least one signer is human or where real-time response is not required. The lack of performance is likely due to an older variant [33] of threshold signature used within the experiments. Newer versions of threshold ECDSA such as DKLS [25], that run in three rounds compared to ten, could significantly improve the performance. It should be noted that threshold variants are being actively researched and more performant solution may emerge.

While the evaluation was conducted for the X-Road data exchange system, the approach can be applied to other data exchange systems, where ODI is verified through Public Key Infrastructure (PKI), e.g., Dutch NLX [67]. Even if the system is not prone to a single point of failure but lacks zero trust, the proposed DKMS enable ODI usage policy enforcement that can cryptographically guarantee the authenticity of the data exchanges.

Finally, though the study has been conducted in the context of the problem of centrally issued PKI certificates and certificate-based key management, the review of mechanisms and system characteristics is agnostic to the trust model. Thus, the proposed DKMS design can be used for self-sovereign identity-based organisational digital identity where verifiable credentials are issued through a distributed ledger but based on the decentralised public key infrastructure [10].

Acknowledgements This paper is supported by the European Union under Grant Agreement No. 101087529 (CHESS).

Author Contributions M.B., P.Š., and R.M. were responsible for the conceptualisation and methodology of the research. M.B. and J.K. conducted the investigation, searched the resources, and wrote the main manuscript text. All authors reviewed the manuscript and were involved in editing. M.B. prepared figures for the visualisation and managed the project as a result of which this paper was written. J.K. developed the software and conducted the validation. P.Š. and R.M. were the supervisors.

Funding Open access publishing supported by the institutions participating in the CzechELib Transformative Agreement.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Abidin, A., Aly, A., Mustafa, M.A.: Collaborative authentication using threshold cryptography. In: Saracino, A., Mori, P. (eds.) *Emerging Technologies for Authorization and Authentication*, pp. 122–137. Springer International Publishing, Cham (2020)
2. Abraham, A., Koch, K., More, S., Ramacher, S., Stopar, M.: Privacy-preserving eID derivation to self-sovereign identity systems with offline revocation, pp. 506–513. IEEE (2021)
3. Alkeilani Alkadri, N., Das, P., Erwig, A., Faust, S., Krämer, J., Riahi, S., Struck, P.: Deterministic wallets in a quantum world. In: *CCS 2020*, p. 1017–1031. ACM (2020). <https://doi.org/10.1145/3372297.3423361>
4. Almansa, J.F., Damgård, I., Nielsen, J.B.: Simplified threshold RSA with adaptive and proactive security. In: Vaudenay, S. (ed.) *Advances in Cryptology - EUROCRYPT 2006*, pp. 593–611. Springer, Berlin Heidelberg, Berlin, Heidelberg (2006)
5. Bacho, R., Das, S., Loss, J., Ren, L.: Adaptively secure three-round threshold schnorr signatures from DDH. *Cryptology ePrint Archive*, Paper 2025/1009 (2025). <https://eprint.iacr.org/2025/1009>
6. Bacho, R., Das, S., Loss, J., Ren, L.: Glacius: Threshold schnorr signatures from DDH with full adaptive security. In: Fehr, S., Fouque, P.A. (eds.) *Advances in Cryptology - EUROCRYPT 2025*, pp. 304–334. Springer Nature Switzerland, Cham (2025)
7. Bakhtina, M.: A method for information security and privacy management in smart solutions. Ph.D. thesis, University of Tartu (2025). Available at <https://hdl.handle.net/10062/108471>
8. Bakhtina, M., Kvapil, J., Švenda, P., Matulevičius, R.: The power of many: Securing organisational identity through distributed key management. In: Guizzardi, G., Santoro, F., Mouratidis, H., Soffer, P. (eds.) *Advanced Information Systems Engineering*, pp. 475–491. Springer Nature Switzerland, Cham (2024)
9. Bakhtina, M., Kvapil, J., Svenda, P., Matulevičius, R.: Review of key management mechanisms (2024). <https://doi.org/10.5281/zenodo.10886209>
10. Bakhtina, M., Leung, K.L., Matulevičius, R., Awad, A., Švenda, P.: A decentralised public key infrastructure for X-Road. In: *ARES 2023*, ACM (2023)
11. Bakhtina, M., Matulevičius, R., Awad, A., Kivimäki, P.: On the shift to decentralised identity management in distributed data exchange

- systems. In: SAC 2023, p. 864–873. ACM (2023). <https://doi.org/10.1145/3555776.3577678>
12. Bamasak, O., Zhang, N., Edwards, D.: Disigncrypton: an integration of agent-based signature delegation with distributed reputation management scheme. In: 19th IEEE International Parallel and Distributed Processing Symposium, pp. 8 pp.– (2005). <https://doi.org/10.1109/IPDPS.2005.169>
 13. Barak, B., Herzberg, A., Naor, D., Shai, E.: The proactive security toolkit and applications. In: Proceedings of the 6th ACM Conference on Computer and Communications Security, CCS '99, p. 18–27. Association for Computing Machinery, New York, NY, USA (1999). <https://doi.org/10.1145/319709.319713>. <https://doi.org/10.1145/319709.319713>
 14. Barker, E.: NIST SP 800-57. Recommendation for key management (2016)
 15. Beerliová-Trubíniová, Z., Hirt, M., Nielsen, J.B.: On the theoretical gap between synchronous and asynchronous mpc protocols. In: Proceedings of the 29th ACM SIGACT-SIGOPS Symposium on Principles of Distributed Computing, PODC '10, p. 211–218. Association for Computing Machinery, New York, NY, USA (2010). <https://doi.org/10.1145/1835698.1835746>. <https://doi.org/10.1145/1835698.1835746>
 16. Boneh, D., Komlo, C.: Threshold signatures with private accountability. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology - CRYPTO 2022, pp. 551–581. Springer Nature Switzerland, Cham (2022)
 17. Buck, C., Olenberger, C., Schweizer, A., Völter, F., Eymann, T.: Never trust, always verify: A multivocal literature review on current knowledge and research gaps of zero-trust. Computers & Security **110**, 102436 (2021)
 18. Cachin, C., Kursawe, K., Lysyanskaya, A., Stroh, R.: Asynchronous verifiable secret sharing and proactive cryptosystems. In: Proceedings of the 9th ACM Conference on Computer and Communications Security, CCS '02, p. 88–97. Association for Computing Machinery, New York, NY, USA (2002). <https://doi.org/10.1145/586110.586124>
 19. Canetti, R., Gennaro, R., Goldfeder, S., Makriyannis, N., Peled, U.: UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, CCS '20, p. 1769–1787. Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3372297.3423367>
 20. Cybernetica: Unified eXchange Platform (UXP). <https://cyber.ee/>
 21. Damgård, I., Jakobsen, T.P., Nielsen, J.B., Pagter, J.I., Østergård, M.B.: Fast threshold ECDSA with honest majority. Cryptology ePrint Archive, Paper 2020/501 (2020). <https://eprint.iacr.org/2020/501>
 22. Damgård, I., Koprowski, M.: Practical Threshold RSA Signatures without a Trusted Dealer. In: B. Pfitzmann (ed.) Advances in Cryptology — EUROCRYPT 2001, pp. 152–165. Springer, Berlin, Heidelberg (2001). https://doi.org/10.1007/3-540-44987-6_10
 23. Das, P., Erwig, A., Faust, S., Loss, J., Riahi, S.: The exact security of bip32 wallets. In: CCS 2021, p. 1020–1042. ACM (2021)
 24. Di Crescenzo, G., Ferguson, N., Impagliazzo, R., Jakobsson, M.: How to forget a secret. In: Meinel, C., Tison, S. (eds.) STACS 99, pp. 500–509. Springer, Berlin Heidelberg, Berlin, Heidelberg (1999)
 25. Doerner, J., Kondi, Y., Lee, E., Shelat, A.: Threshold ECDSA in Three Rounds . In: 2024 IEEE Symposium on Security and Privacy (SP), pp. 3053–3071. IEEE Computer Society, Los Alamitos, CA, USA (2024). <https://doi.org/10.1109/SP54263.2024.00178>. <https://doi.ieeecomputersociety.org/10.1109/SP54263.2024.00178>
 26. European Commission: Preparatory work in view of the procurement of an open source cloud-to-edge middleware platform, (2022)
 27. Fan, C.I., Tseng, Y.F., Su, H.P., Hsu, R.H., Kikuchi, H.: Secure hierarchical bitcoin wallet scheme against privilege escalation attacks. Int. J. Inf. Secur. **19**(3), 245–255 (2020). <https://doi.org/10.1007/s10207-019-00476-5>
 28. Frankel, Y., Gemmell, P., MacKenzie, P., Yung, M.: Optimal-resilience proactive public-key cryptosystems. In: Proceedings 38th Annual Symposium on Foundations of Computer Science, pp. 384–393 (1997). <https://doi.org/10.1109/SFCS.1997.646127>
 29. Frankel, Y., Gemmell, P., MacKenzie, P.D., Yung, M.: Proactive RSA. In: Kaliski, B.S. (ed.) Advances in Cryptology – CRYPTO '97, pp. 440–454. Springer, Berlin Heidelberg, Berlin, Heidelberg (1997)
 30. Frankel, Y., MacKenzie, P., Yung, M.: Adaptively-secure optimal-resilience proactive RSA. In: Lam, K.Y., Okamoto, E., Xing, C. (eds.) Advances in Cryptology - ASIACRYPT'99, pp. 180–194. Springer, Berlin Heidelberg, Berlin, Heidelberg (1999)
 31. Gaia-X European Association for Data and Cloud AISBL: Gaia-X: A Federated Secure Data Infrastructure. <https://gaia-x.eu/>
 32. Gennaro, R., Goldfeder, S.: Fast Multiparty Threshold ECDSA with Fast Trustless Setup. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS '18, pp. 1179–1194. Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3243734.3243859>. <https://doi.org/10.1145/3243734.3243859>
 33. Gennaro, R., Goldfeder, S.: Fast multiparty threshold ecdsa with fast trustless setup. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS '18, p. 1179–1194. Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3243734.3243859>. <https://doi.org/10.1145/3243734.3243859>
 34. GitHub: Deploying threshold signatures into X-Road. <https://github.com/crocs-muni/xroad-threshold-signatures>
 35. GitHub: Pretzel. <https://github.com/crocs-muni/pretzel>
 36. Guthoff, C., Anell, S., Hainzinger, J., Dabrowski, A., Krombholz, K.: Perceptions of distributed ledger technology key management – an interview study with finance professionals. In: IEEE SP 2023, pp. 588–605 (2023)
 37. Herzberg, A., Jakobsson, M., Jarecki, S., Krawczyk, H., Yung, M.: Proactive public key and signature systems. In: Proceedings of the 4th ACM Conference on Computer and Communications Security, CCS '97, p. 100–110. Association for Computing Machinery, New York, NY, USA (1997). <https://doi.org/10.1145/266420.266442>. <https://doi.org/10.1145/266420.266442>
 38. Herzberg, A., Jarecki, S., Krawczyk, H., Yung, M.: Proactive secret sharing or: How to cope with perpetual leakage. In: Coppersmith, D. (ed.) Advances in Cryptology – CRYPTO' 95, pp. 339–352. Springer, Berlin Heidelberg, Berlin, Heidelberg (1995)
 39. Herzberg, A., Jarecki, S., Krawczyk, H., Yung, M.: Proactive secret sharing or: How to cope with perpetual leakage. In: Annual international cryptology conference, pp. 339–352. Springer (1995)
 40. Hevner, A.R., March, S.T., Park, J., Ram, S.: Design science in information systems research. MIS Q. **28**(1), 75–105 (2004)
 41. Jain, M., Singh, M.: Identity based secure RSA encryption system. In: N. Modi, P. Verma, B. Trivedi (eds.) Proceedings of International Conference on Communication and Networks, pp. 429–437. Springer Singapore, Singapore (2017)
 42. Jarecki, S., Olsen, J.: Proactive RSA with non-interactive signing. In: Tsudik, G. (ed.) Financial Cryptography and Data Security, pp. 215–230. Springer, Berlin Heidelberg, Berlin, Heidelberg (2008)
 43. Johnson, D., Menezes, A., Vanstone, S.A.: The elliptic curve digital signature algorithm (ECDSA). Int. J. Inf. Sec. **1**(1), 36–63 (2001)
 44. Katsumata, S., Reichle, M., Takemure, K.: Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology - CRYPTO 2024, pp. 459–491. Springer Nature Switzerland, Cham (2024)

45. Kersic, V., Vidovic, U., Vrecko, A., Domajnko, M., Turkanovic, M.: Orchestrating digital wallets for on- and off-chain decentralized identity management. *IEEE Access* **11**, 78135–78151 (2023). <https://doi.org/10.1109/ACCESS.2023.3299047>
46. Komlo, C.: Threshold signatures. *IEEE Security & Privacy* **22**(6), 85–88 (2024). <https://doi.org/10.1109/MSEC.2024.3463915>
47. Komlo, C., Goldberg, I.: Frost: Flexible round-optimized schnorr threshold signatures. In: *Selected Areas in Cryptography*, pp. 34–65. Springer, Cham (2021)
48. Kondi, Y., Magri, B., Orlandi, C., Shlomovits, O.: Refresh when you wake up: Proactive threshold wallets with offline devices. *Cryptology ePrint Archive, Paper 2019/1328* (2019). <https://eprint.iacr.org/2019/1328>
49. Krimmer, R., Dedovic, S., Schmidt, C., Corici, A.A.: Developing cross-border e-governance: Exploring interoperability and cross-border integration. In: *Electronic Participation*, pp. 107–124. Springer, Cham (2021)
50. McBride, K., Kamalanathan, S., Valdma, S.M., Toomere, T., Freudenthal, M.: Digital government interoperability and data exchange platforms: Insights from a twenty country comparative study. In: *ICEGOV 2022*, p. 90–97. ACM (2022)
51. Nair, V., Song, D.: Decentralizing custodial wallets with MFKDF. In: *IEEE ICBC 2023*, pp. 1–9 (2023). <https://doi.org/10.1109/ICBC56567.2023.10174998>
52. Nair, V., Song, D.: Multi-Factor key derivation function (MFKDF) for fast, flexible, secure, & practical key management. In: *32nd USENIX Security Symposium (USENIX Security 23)*, pp. 2097–2114. USENIX Association, Anaheim, CA (2023). <https://www.usenix.org/conference/usenixsecurity23/presentation/nair-mfkdf>
53. N'goran, K.R., Brou, A.P.B., Pandry, K.G., Tetchueng, J.L., Kermarrec, Y., Asseu, O.: Zero trust security strategy for collaboration systems. In: *2023 International Symposium on Networks, Computers and Communications (ISNCC)*, pp. 1–6 (2023). <https://doi.org/10.1109/ISNCC58260.2023.10323911>
54. NIIS: X-Road Documentation. <https://docs.x-road.global/>
55. NIIS: X-ROAD®. <https://x-road.global/>
56. Pieter Wuille: BIP 0032. Hierarchical Deterministic Wallets. <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>
57. Preukschat, A., Reed, D.: *Self-sovereign identity*. Manning Publications, Shelter Island, NY (2021)
58. Rabin, T.: A simplified approach to threshold and proactive RSA. In: Krawczyk, H. (ed.) *Advances in Cryptology – CRYPTO '98*, pp. 89–104. Springer, Berlin Heidelberg, Berlin, Heidelberg (1998)
59. Rivest, R.L., Shamir, A., Adleman, L.: A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM* **21**(2), 120–126 (1978). <https://doi.org/10.1145/359340.359342>
60. Rose, S., Borchert, O., Mitchell, S., Connelly, S.: NIST SP 800-207. Zero trust architecture (2020)
61. Sarfaraz, A., Chakraborty, R.K., Essam, D.L.: AccessChain: An access control framework to protect data access in blockchain enabled supply chain. *FGCS* **148**, 380–394 (2023). <https://doi.org/10.1016/j.future.2023.06.009>
62. Scarlata, M., Backendal, M., Haller, M.: MFKDF: Multiple factors knocked down flat. In: *33rd USENIX Security Symposium (USENIX Security 24)*, pp. 4301–4318. USENIX Association, Philadelphia, PA (2024). <https://www.usenix.org/conference/usenixsecurity24/presentation/scarlata>
63. Shoup, V.: Practical threshold signatures. In: *Advances in Cryptology – EUROCRYPT 2000*, pp. 207–220. Springer Berlin Heidelberg, Berlin, Heidelberg (2000)
64. Soltani, R., Nguyen, U.T., An, A.: Decentralized and privacy-preserving key management model. In: *ISNCC 2020*, pp. 1–7. (2020)
65. Tan, S.Y., Heng, S.H.: New identity-based identification and signature schemes in the standard model. In: *ARES 2023*. ACM (2023). <https://doi.org/10.1145/3600160.3604999>
66. Verizon Business: 2023 data breach investigations report, (2023)
67. VNG Realisatie: NLX: Documentation. <https://docs.fsc.nlx.io/>
68. Windley, P.J.: *Learning Digital Identity: Design, Deploy, and Manage Identity Architectures*. O'Reilly Media, Incorporated (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.